پژوهش‌سرای دانش‌آموزی امام هادی (ع)

نشریهٔ کدنویسی پژوهش‌سرا

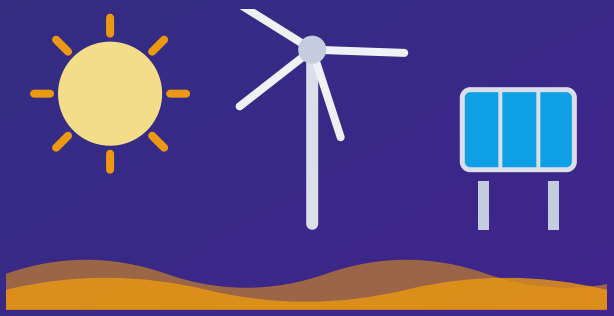
ویژهٔ آبان ۱۴۰۵ · شمارهٔ ۲

مسیر Rust از صفر به ۱۴ درس رسید؛ حالا نوبت توست بسازی!

درسنامهٔ کامل درس ۱۳ و ۱۴ · آزمون چهارگزینه‌ای · چالش برنامه‌نویسی آبان · خوارزمی و هاپر

درسنامهٔ کامل درس ۱۳ و ۱۴ | آزمون چهارگزینه‌ای | چالش برنامه‌نویسی | مسیر کامل ۱۴ درس

```
fn main() {    println!("Salam, Aban 1405!");}
```



نشریه‌های پژوهش‌سرا
قابل چاپ · رایگان · دانش‌آموزی

ble.ir/anjomanfaghihi

قلمِ اول سرمقاله

آبان امسال برای پژوهش‌سرا ماهِ تمام‌شدن یک مسیر بود؛ و شروعِ مسیری تازه.

درست همان‌طور که قول داده بودیم، مسیر «Rust از صفر» با درس سیزدهم و چهاردهم کامل شد. الان هر خواننده‌ای که هر چهارده درس را خوانده باشد، از «Rust چیست؟» تا «بردار و فهرست کارها» راه را از اول تا آخر رفته است. این چهارده درس سه‌ونیم ماه دوید؛ درس‌به‌درس، با مثال و تمرین و پاسخ.

اما تمام‌شدن درس‌ها پایان راه نیست؛ شروع است. زبان فقط ابزار است و ابزار وقتی ارزش دارد که با آن چیزی ساخته شود. برای همین درسنامهٔ ویژهٔ این شماره، هر دو درس تازه را کامل باز کرده و تمرین بزرگش یک پروژهٔ واقعی است: «فهرست کارها» را خودت بنویس — همان برنامه‌ای که در درس ۱۴ قدم‌به‌قدم ساختیم، این بار با ایده‌های خودت.

اگر جا موندی، نگران نباش: پوستر و درسنامهٔ هر چهارده درس در کانال کدنویسی منتشر شده و هر درس با مثال و تمرین همراه است. و اگر سؤال یا ایده‌ای داری، در کانال بفرست — نام تو کنار مطلبش چاپ می‌شود.

چه گذشت؟ خبرنامهٔ پژوهش‌سرا

1. **انتشار درس ۱۳ — شمارشی‌ها (enum) و match**
   با پروندهٔ کامل در همین شماره؛ چراغ روشن و خاموش، اولین enum زندگی‌ات.

2. **انتشار درس ۱۴ — بردار (Vec) و پروژهٔ «فهرست کارها»**
   با این درس، مسیر ۱۴ درسی «Rust از صفر» کامل شد.

3. **چالش برنامه‌نویسی آبان**
   با همان آموخته‌های ۱۴ درس یک برنامهٔ کوچک واقعی بنویس و در کانال به اشتراک بگذار — جزئیات در صفحهٔ چالش.

4. **برنامهٔ ثابت هفتگی**
   درس‌های تازه هر هفته در کانال کدنویسی؛ تمرین‌ها و پاسخ‌ها در شمارهٔ بعدی نشریه.

درسنامهٔ ویژه · بخش اول

درس ۱۳ — شمارشی‌ها (enum) و تطبیق الگو با match

تا حالا متغیرهایمان عدد و متن بودند؛ ولی دنیای واقعی «حالت» هم دارد: چراغ یا روشن است یا خاموش، بازی یا برنده است یا بازنده، روز هفته یا شنبه است یا چهارشنبه. برای همین داده‌های Rust ابزار قشنگی دارد: enum (شمارشی) — قالبی که می‌گویی «این مقدار فقط یکی از این چند حالت را می‌تواند باشد».

همنشین enum کلیدواژهٔ match است: مثل تابلوی برق چندتایی، هر حالت را می‌گیری و برایش کارِ خودش را می‌کنی. نکتهٔ طلایی: کامپایلر مواظب توست — اگر یک حالت را جا بیندازی، برنامه اصلاً ساخته نمی‌شود! یعنی Rust اجازه نمی‌دهد حالتی از قلم بیفتد و بعداً وسط کارِ کاربر سرکش کند.

فایل «cheragh.rs» — اولین شمارشیِ تو

```
enum Cheragh {
    Roshan,
    Khamoosh,
}

fn main() {
    let cheragh = Cheragh::Roshan;

    match cheragh {
        Cheragh::Roshan => println!("Roshan!"),
        Cheragh::Khamoosh => println!("Khamoosh."),
    }
}
```

نکتهٔ طلایی	یعنی چه؟
enum فقط چند حالت دارد	مقدارِ نوعِ Cheragh فقط می‌تواند Roshan یا Khamoosh باشد — چیزی سوم در کار نیست.
match همهٔ حالت‌ها را می‌پوشاند	هر حالت یک شاخه؛ هر شاخه یک کار. مثل تابلوی برقی که هر کلید یک مدار را وصل می‌کند.
حالتِ جامانده = خطای کامپایل	کامپایلر خودش شماره می‌گذارد: «شاخهٔ Khamoosh را نوشتی؟» — باگ قبل از کاربر گرفته می‌شود.

تمرین درس ۱۳: یک enum برای فصل‌های سال بساز و با match برای هر فصل یک یادگاری چاپ کن — بهار: باران و شکوفه، زمستان: برف و نارنج! جوابت را در کانال بفرست.

درسنامهٔ ویژه · بخش دوم

درس ۱۴ — بردار (Vec) و پروژهٔ «فهرست کارها»

آرایه را یادت هست؟ طولش از اول ثابت بود. ولی فهرست خرید تو هر روز عوض می‌شود؛ یکی اضافه می‌شود، یکی خط می‌خورد. اینجا بردار (Vec) به کمکمان می‌آید: لیستی که نفس می‌کشد — با push بزرگ می‌شود، اندازه‌اش هر چقدر که لازم باشد.

چون قرار است وسط برنامه عوضش کنیم، کنار let کلمهٔ mut می‌گذاریم؛ مخفف mutable یعنی «این جعبه قرار است تغییر کند». با Vec::new() بردار تازه می‌سازیم، با push عضو اضافه می‌کنیم و با len طولش را می‌پرسیم؛ بعد با همان for درس‌های قبل روی همهٔ اعضایش قدم می‌زنیم.

و حالا پروژهٔ کوچکِ آخر مسیر: سازهٔ کار (Kar) از درس ۱۲ + بردار = «فهرست کارها»! کار اضافه کن، انجام‌شده‌ها را علامت بزن، همه را چاپ کن. اگر همین برنامهٔ کوچک را خودت بنویسی، یعنی از درس ۱ تا ۱۴ را واقعاً یاد گرفته‌ای.

فایل «fehrest-kar.rs» — بخش اول: کارِ تازه

```
struct Kar {
    onvan: String,
    anjamShod: bool,
}

fn main() {
    let mut list: Vec<Kar> = Vec::new();

    list.push(Kar {
        onvan: String::from("tamrin-e rust"),
        anjamShod: false,
    });
```

بخش دوم: چاپ فهرست

```
    list.push(Kar {
        onvan: String::from("khandan-e dars 14"),
        anjamShod: true,
    });

    println!("tedad-e karha: {}", list.len());

    for kar in list {
        if kar.anjamShod {
            println!("[x] {}", kar.onvan);
        } else {
            println!("[ ] {}", kar.onvan);
        }
    }
}
```

ابزار بردار	کارش چیست؟
Vec::new()	یک بردار خالی می‌سازد؛ هنوز هیچ عضوی ندارد.
list.push(kar)	یک عضو به انتهای بردار اضافه می‌کند؛ بردار خودش جا باز می‌کند.
list.len()	طول بردار را می‌دهد؛ فعلاً تعداد کارها.
let mut list	بدون mut اجازهٔ push و تغییر نداری — Rust اول اجازه می‌خواهد.

خودت را بسنج — آزمون چهارگزینه‌ای

شش پرسش از دو درس تازه؛ پاسخ‌ها پایین صفحه است — اول خودت جواب بده!

۱. enum برای چیست؟

ا. وقتی مقدار فقط می‌تواند یکی از چند حالت مشخص باشد

ل. نگه‌داشتن فهرست خرید

ف. تکرار کد

ب. پاک کردن حافظه

۲. اگر در match یکی از حالت‌ها را جا بیندازیم چه می‌شود؟

ا. کامپایل خطا می‌دهد و برنامه ساخته نمی‌شود

ل. برنامه همان را نادیده می‌گیرد

ف. کامپایلر حدس می‌زند

ب. هیچ اتفاقی نمی‌افتد

۳. match چه‌طور کار می‌کند؟

ا. هر حالت را با یک شاخه مقایسه می‌کند و کارِ همان شاخه را می‌کند

ل. همیشه اولین شاخه را اجرا می‌کند

ف. فقط با عدد کار می‌کند

ب. متن را پاک می‌کند

۴. فرق Vec با آرایه چیست؟

ا. طول Vec عوض می‌شود؛ آرایه ثابت است

ل. Vec فقط متن نگه می‌دارد

ف. آرایه فقط عدد می‌گیرد

ب. فرقی ندارند

۵. به بردار چطور عضو اضافه می‌کنیم؟

ا. با push

ل. با add

ف. با plus

ب. با give

۶. چرا کنار let کلمهٔ mut می‌گذاریم؟

ا. چون قرار است مقدارش وسط برنامه عوض شود

ل. چون Rust خیلی دوستش دارد

ف. برای سریع‌تر شدن

ب. لازم نیست؛ فقط قشنگ است

پاسخ‌ها: ۱-الف · ۲-الف · ۳-الف · ۴-الف · ۵-الف · ۶-الف — همهٔ پاسخ‌های درست گزینهٔ اول‌اند؛ پیش از دیدن این خط جواب دادی؟ آفرین!

نقشهٔ راه مسیر کامل ۱۴ درسِ Rust — یک نگاه

#	درس	کلیدواژه
۱	Rust چیست و چرا این زبان؟	معرفی
۲	نصب و اولین برنامه	cargo new
۳	چاپ متن روی صفحه	!println
۴	کامنت‌ها و پیام‌های کامپایلر	//
۵	متغیرها	let · let mut
۶	انواع داده	i32 · f64
۷	رشته‌ها؛ دوختن متن و عدد	String
۸	تصمیم‌گیری در برنامه	if
۹	حلقه‌ها؛ تکرار بدون خستگی	loop · while
۱۰	آرایه و پیمایش فهرست اعداد	for
۱۱	توابع با return	fn
۱۲	سازه‌ها	struct
۱۳	شمارشی‌ها و تطبیق الگو	enum · match
۱۴	بردار و فهرست کارها	Vec · push

پشت صحنهٔ خطاها پرونده: چرا Rust از همه سخت‌گیرتر است؟

در طول این چهارده درس بارها دیدی که کامپایلر Rust خطا می‌دهد؛ حتی وقتی کد «به نظر» درست می‌آید. مثلاً وقتی می‌خواهی مقدار متغیر قفل‌شده را عوض کنی، یا جعبهٔ عددی را با متن پر کنی. چرا Rust این‌قدر سخت می‌گیرد؟ جوابش سه راز دارد.

مالکیت (Ownership)

هر تکه‌داده یک مالک روشن دارد. وقتی مالک تمام شود، حافظه خودبه‌خود آزاد می‌شود — بدون نشتی حافظه. برای همین Rust هم سریع است و هم امن.

قرض‌دادن (Borrowing)

می‌شود داده را به تابع «قرض» داد تا فقط بخواند یا با اجازه عوض کند. Rust حساب‌دار دقیقی است: هم‌زمان نمی‌گذارد دو نفر یکی را عوض کنند.

تغییرناپذیری پیش‌فرض

یادت هست درس ۵؟ متغیر پیش‌فرض قفل است و تا ننویسی let mut تغییر نمی‌کند. همین سخت‌گیری برنامه را قابل‌پیش‌بینی نگه می‌دارد.

وقتی کامپایلر Rust خطا می‌دهد، در حال دفاع از توست: هر error یک باگ است که قبل از رسیدن به دست کاربر گرفته شده. برنامه‌نویس‌های حرفه‌ای به پیام‌های کامپایلر «دوستِ صمیمی» می‌گویند!

چالش برنامه‌نویسی آبان هنوز وقت داری!

چالش جاری کانال: خروجی این برنامه چیست؟

چالش — خروجی را حدس بزن

```
fn main() {
    let javab = 2 + 3 * 4;
    println!("javab: {}", javab);
}
```

راهنما: ترتیب عملگرها مثل ریاضیِ مدرسه است — اول ضرب، بعد جمع. حدست را یادداشت کن و جوابت را در کانال بفرست؛ پاسخ رسمی در چالش بعدی اعلام می‌شود. پاسخ‌های درست در شمارهٔ آذر با نام خودت چاپ می‌شوند.

مشاهیر؛ نقطهٔ شروع از ایران پروندهٔ تاریخ کد



خوارزمی — پدر الگوریتم



گریس هاپر — مادر کامپایلر

خوارزمی، ریاضی‌دان ایرانی سدهٔ نهم؛ واژهٔ «الگوریتم» از نام او ساخته شد و کتاب «الجبر»ش منشأ واژهٔ جبر در ریاضیات جهان است. هر بار که «الگوریتم» می‌نویسی، یک نام ایرانی روی صفحهٔ توست.

گریس هاپر، «مادر کامپایلر»؛ نخستین کامپایلر تاریخ را ساخت تا برنامه‌نویس بتواند به‌جای کدِ ماشین، به زبانی نزدیک به زبان خودِ انسان بنویسد؛ کاری که مسیر زبان‌های امروزی — از C تا Rust — را باز کرد.

آمارِ پژوهش‌سرا جمع‌بندی این شماره

۱۴	۶	۱	۲
درسِ مسیر Rust از صفر	پرسش آزمون این شماره	چالش باز تا شمارهٔ آذر	درسنامهٔ ویژه: enum و Vec

پایان آبان — به زبان Rust

```
fn entezar() {
    let aban = "tamam shod";
    let azar = "hanooz dar rah";
    (aban, azar)
}
```

کانال کدنویسی پژوهش‌سرا

مسیر Rust از صفر، چالش‌ها، معرفی مشاهیر و کتاب — هر هفته یک درس تازه. جا موندی؟ از درس ۱ شروع کن؛ هر چهارده درس با مثال، تمرین و پوستر در کانال منتشر شده است.

ble.ir/anjomanfaghihi

شمارهٔ آذر: پاسخ چالش آبان، تمرین‌های تازه و مطلبِ شما — بفرست تا با نامت چاپ شود.